

Learning to choose, then navigate

Experiment 2 asks whether an explicitly trained controller can associate a visual pattern with reward and learn routes to food in an editable maze. A two-dimensional fly avatar makes the decisions visible. This is an engineering experiment that uses a fixed connectome-derived sensory encoder; it is not a claim that biological fly memories have been reconstructed.

Prospective status

Written on 20 September 2026 before implementation of this learning mechanism and before its training outcomes were observed. Numerical verification and exploratory development follow. This is a dated local protocol, not an external preregistration or a guarantee of success.

Preserve Experiment 1

Experiment 1 keeps its current launcher, UI, dynamics, graph and PDFs. The local Git tag exp-1-v0.2 points to commit 53dff10. Experiment 2 has setup_exp_2.sh, its own service and port 8767, separate run records and an independent interface. The existing Mac PLAN folder gains only this exp_2.pdf.

The controller learns. Connectome edges and neural model parameters remain fixed. A successful food-finding route alone does not demonstrate biological learning, cognitive understanding or an advantage from the fly wiring.

The first maze task

The arena is a 9 by 9 grid. Walls block movement. The user can edit walls, the starting cell and two food sites. The agent has four actions: north, east, south and west. One action moves one grid cell. The rendered body is a schematic avatar, not a biomechanical fly.

The two sites display Pattern A (vertical bars) and Pattern B (horizontal bars). Their assignment swaps across episodes using a reproducible, balanced schedule. The rewarded pattern is configured before the experiment; the controller is not given the reward label. Contact with either food site ends the episode.

A correct food contact yields +1; the other pattern yields -0.25. Each ordinary movement costs -0.01. An attempted manual move into a wall costs -0.04. Autonomous policies see adjacent walls and choose among legal moves. The episode times out after 120 actions.

What the agent observes

Its engineered grid position, legal adjacent movements, and both food-site pattern signals are available throughout an episode. The visual beacons are readable across the arena and are not occluded by walls. The controller receives decoded sensory tokens; it receives no shortest path, food payoff label or distance-to-goal oracle.

This first task is route learning with visible beacons and perfect grid localization. It does not yet model an unknown maze, realistic fly vision, smell, fog of war or a general search strategy. Transfer tests explicitly check the limits of route memory.

From patterns to neural signals

In real-data mode, load the same 166,700-node MaleCNS-derived graph and fixed LIF equations used in Experiment 1. Feed a 32 by 24 pattern into mapped L1 columns on both sides, using the existing exploratory coordinate mapping and a drive of 0.6 mV per tick. Each pattern probe starts from reset and runs for 40 ms.

Pool measured L1 spike rates into a 4 by 4 grid for each eye: 32 neural features. Preserve the feature vectors, whole-network counts, voltage summaries, mapping identity, model parameters and graph checksums. Demonstrate distinguishable, finite responses before allowing neural-cue training.

The two known pattern responses become fixed calibration templates. Classification chooses the nearest response template. This is an engineered decoder calibrated on the two stimuli; it is not a discovered natural motor or recognition circuit.

Exact cached probes, not a continuous brain

Because this task repeats only two deterministic images, compute each reset response once and reuse its recorded features by image identity. The UI labels the neural plot as a cached probe. No new full-brain simulation occurs on every movement, and no continuous neural state connects maze steps.

On a Mac the portable default uses the explicitly synthetic teaching graph and CPU solver. On gx10-b the actual MaleCNS probes run on CUDA. The small Q-learning controller and maze run on the same server CPU. Never display cached traces as current whole-brain activity.

The learning rule

Use tabular Q-learning. $Q(s,a)$ estimates the discounted future return of action a in state s . Here s contains the grid position and the decoded pattern at each site. The Q table has four sensory combinations, 81 positions and four actions. Walls and the reward rule are fixed within a training experiment.

One-step update

$Q(s,a)$ becomes $Q(s,a) + \alpha * [r + \gamma * \max Q(\text{next_s}, \text{legal_action}) - Q(s,a)]$. For a terminal food contact or timeout, use r alone as the target. The learned quantities are entries in this external table; no synapse in the connectome is updated. [1]

Defaults: $\alpha = 0.25$; $\gamma = 0.95$. Exploration probability ϵ is $\max(0.05, 0.8 * 0.99^{\text{episode}})$. During training, choose a uniformly random legal action with probability ϵ ; otherwise choose a maximum- Q legal action, breaking ties reproducibly at random. Evaluation uses ϵ zero and never updates Q .

The step cost encourages shorter successful routes without revealing their direction. Exploration can initially look inefficient. A finite run with a fixed learning rate does not inherit an unconditional mathematical guarantee of optimality.

Resetting a trial preserves Q . Starting a new experiment or applying edited geometry starts fresh Q tables and a new run record. This prevents mixing incompatible reward tasks in one learning curve.

Controls that matter

The interface shows an editable maze, a six-legged fly avatar, its trajectory, visited cells and food patterns. The side panels show current reward, episode, exploration, action values, neural template features and the learning curve.

Train, Pause and Step

Train adds a chosen number of episodes to a bounded training queue. Speed controls display pacing in actions per second, not a biological time constant or a learning parameter. Pause keeps Q and the current trial. Step makes one learner decision and applies its update.

Manual movement

The arrow controls move the avatar while paused. Manual trials do not update the trained Q table or enter the autonomous training-success metric. Reset trial starts a clean autonomous trial while retaining learned Q. Finish/save preserves the experiment checkpoint.

Edit the arena

Select Wall, Erase, Start, Food site 1 or Food site 2, then click cells. Draft edits take effect only when Apply new experiment is pressed. Validation requires a closed boundary, distinct open markers and routes from the start to both sites without needing to pass through the other terminal site.

New experiment and reward reversal

New maze drafts a different layout. The rewarded-pattern selector and seed apply when creating a new experiment. Changing the reward identity resets learning; studying reversal while retaining memory requires a separate future protocol amendment.

Controlled evaluation

Every training episode has a matching image-cue control episode. That controller sees the same grid position and legal movements and uses the same Q-learning rule, initial values, exploration schedule, reward scheme and episode random seed. It recognizes the two rendered patterns directly, bypassing the neural simulation.

The image-cue control receives equivalent pattern information. If its decoded tokens match the neural templates, equal performance is expected. That outcome means the maze task and Q-learning worked, without evidence that the connectome contributed an advantage.

Evaluate freezes a policy checkpoint and runs all three agents: trained neural-cue Q, trained image-cue Q, and uniformly random legal movement. Use a disjoint seed namespace for evaluation. Report the trained maze separately from newly generated mazes with the same grid size and marker conventions.

Default evaluation uses 30 episodes per environment group and per agent. Repeat food-pattern swaps in balanced pairs. No weight changes occur during either familiar-maze or unseen-maze evaluation. Evaluation results cannot be appended to the training curve as if they were training trials.

Unseen mazes may expose strong overfitting because Q is indexed by coordinates. Failure to transfer is a valid result. This first protocol cannot establish a universal or best maze-scanning strategy.

What to measure

Primary outcomes: fraction of episodes reaching the rewarded pattern and mean return across all episodes. Also report steps, food mistakes and timeouts. Show a rolling training success rate using the last 20 completed autonomous episodes; label its actual sample count.

During evaluation, preserve per-trial outcomes and report successes divided by n with a Wilson 95 percent interval. This interval describes the finite trial sample, not variation across training seeds or hardware. A single training seed is an exploratory demonstration.

For successful episodes, shortest-path efficiency is shortest allowed start-to-goal length divided by steps taken. The shortest-path calculation belongs only to the evaluator; the policy never receives it. Paths through the wrong terminal food cell are excluded.

For a stronger follow-up, use at least five independently trained seeds, summarize per-seed results, freeze hyperparameters before evaluating and test on additional held-out layouts. Do not claim significance from hundreds of steps of the same policy as if each were an independent training experiment.

Development acceptance

The system must demonstrate actual reward-driven Q updates, checkpoint restoration, unchanged Q during evaluation, correctly excluded manual trials and preserved Experiment 1. Record favorable and unfavorable results. Performance comparisons must name the dataset, trained component and control.

Installation and two-machine use

From a cloned project, run `sh setup_exp_2.sh`. This creates or reuses the isolated runtime, prepares the labeled synthetic graph and starts Experiment 2 on `http://127.0.0.1:8767/`. Use `--no-start` to install and verify without launching. It does not start or modify Experiment 1.

For real data on a CPU, use `sh setup_exp_2.sh --dataset male-cns --backend cpu`. For a compatible Linux NVIDIA system, use `--dataset male-cns --backend cuda`. Python 3.11-3.13 and the same pinned dependencies apply. The full-data memory and download requirements remain those in the handbook.

In this lab, keep Experiment 1 on `gx10-a` at port 8765. Run Experiment 2 on `gx10-b` at port 8767 and forward only loopback to the Mac. The machines run separate experiments; no distributed simulation is claimed. Both receive matching source releases.

The launcher accepts `--port`, `--venv`, `--data-dir` and `--runs-dir`. Its default records directory is `runs/exp_2`. A busy port produces an actionable error; existing processes are not terminated. Cached neural probes are generated from the selected graph, never substituted from another dataset.

The user-facing maze is rendered in the Mac browser. Numerical probes and all experiment state are owned by the selected server. Refreshing the browser does not reset learning. Two tabs of one service share the same experiment.

Records, checkpoints and restoration

Each experiment gets a unique directory containing `source_manifest.json`, `manifest.json`, `config.json`, `commands.jsonl`, `episodes.jsonl`, `evaluations.json` and `checkpoint.npz`. Neural probe vectors and summaries are copied into the record with graph and mapping hashes; source code is preserved once per source identity.

The checkpoint stores both learned Q tables, completed episode count, validated layout/configuration and encoder identity. It is saved after each completed episode and on pause/save. Loading verifies dataset/encoder identity and opens a new run linked to the parent checkpoint, preserving the original record.

Checkpoint restoration resumes from a fresh episode at the boundary. It does not restore a partially completed maze trajectory or a continuously running brain, because those are not the state represented by this checkpoint. The saved episode count preserves the exploration schedule.

Each evaluation records its policy digest, seeds, layouts and trial outcomes. Check Q arrays before and after evaluation to reject accidental training leakage. Manual interventions are marked separately. Partial or failed runs retain their evidence and an explicit status.

Reproduction

Preserve source archives, graph data/checksums, neural probe artifacts, seed/configuration and policy checkpoints. A replay of the external controller can reuse verified probe features; reproducing the neuroscience probe additionally needs the graph and numerical backend. Dataset downloads and run records remain outside Git.

Interpretation and next stages

A higher reward rate on the training maze shows a controller adapted to this finite task. It does not show that the MaleCNS organism learned, that fly synapses changed, or that the avatar has natural fly behavior. The interface must keep the trainable controller and fixed neural encoder visibly separate.

A similar result for the image-cue baseline is informative. It identifies the ordinary learner as sufficient for the task. A claimed benefit from fly anatomy would need harder matched tests and appropriate wiring perturbations, with encoder information and training budgets controlled.

After this version: (1) test unseen-layout transfer, (2) add partial visibility and explicit agent memory, (3) consider an olfactory search task, or (4) introduce a stated mushroom-body-inspired plasticity rule. Each addition changes the scientific question and requires a dated amendment before evaluating its claims.

Published embodied fly platforms provide useful examples of sensory interfaces and trained controllers. They do not make our particular graph, dynamics or Q table biologically validated. Connectome-constrained visual work also fits physiological parameters; we are not adopting those fitted dynamics automatically. [2,3]

Success is measured improvement with reproducible records and appropriate controls. If the best observed strategy memorizes this maze, report memorization. If transfer or biological specificity is absent, report that explicitly.

References and protocol boundary

[01] Watkins and Dayan. Q-learning. Machine Learning (1992).

<https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf>

[02] Wang-Chen et al. NeuroMechFly v2: simulating embodied sensorimotor control in adult Drosophila. Nature Methods (2024).

<https://www.nature.com/articles/s41592-024-02497-y>

[03] Lappalainen et al. Connectome-constrained networks predict neural activity across the fly visual system. Nature (2024).

<https://www.nature.com/articles/s41586-024-07939-3>

[04] MaleCNS official download inventory.

<https://male-cns.janelia.org/download/>

The official MaleCNS release supplies anatomical data. The current project supplies assumed dynamics, direct L1 stimulation, calibrated template readout, grid localization, reward values and the external learning mechanism. These layers must remain distinguishable.

Protocol v0.1 is the pre-implementation specification. Implementation checks and observed development results belong in STATUS.md and run records. A later document revision must identify what was observed before it was written. Keep one current exp_2.pdf in PLAN and preserve prior source versions in Git.